AWS HIGH AVAILABILITY WEB APPLICATION

Professional Project Documentation

VPC • Multi-AZ Subnets • Bastion Host • Application Load Balancer • Auto Scaling

Document field	Details
Project title	AWS High Availability Web Application
Student name	[Dipali Ajinkya Nimbalkar]
AWS Region	Asia Pacific (Mumbai) — ap-south-1
VPC name	My Project VPC
VPC ID	vpc-01b9fdae82b418aad
GitHub repository	[https://github.com/dipali-devops95/aws-highly-available-web-app]
Submission date	[Enter date]

Accuracy note: This report follows the known project details from the implementation. Where exact values (such as ALB name, target group name, AMI, listener settings, S3 bucket name, or Route 53 record) are not confirmed, a fill-in field is provided. Enter only the values actually present in your AWS account. This project documentation does not describe Blue/Green deployment.

1. Executive Summary

This project demonstrates a highly available web application architecture on Amazon Web Services (AWS). The environment uses a custom VPC with four subnets across two Availability Zones, an Internet Gateway and route tables, a Bastion Host for controlled administrative access, and an Application Load Balancer with a target group and Auto Scaling Group. The load balancer distributes incoming HTTP/HTTPS requests to healthy application instances, while Auto Scaling helps maintain the configured capacity. The implementation was tested through AWS resource status checks and browser access to the application. The final report should include verified screenshots and actual test outcomes.

2. Project Objectives

- Create a dedicated VPC and divide resources into public and private subnets.
- Use two Availability Zones to support resilient network and application design.
- Configure Internet Gateway, route tables, and subnet associations.
- Create a Bastion Host for controlled access to private resources.
- Configure an Application Load Balancer and target group for application traffic.
- Use a Launch Template and Auto Scaling Group to launch and manage application EC2 instances.
- Validate instance health, target health, and application access through the load balancer.
- Document the implementation and publish sanitized project evidence to GitHub.

3. AWS Services Used

Service / component	Role in the project
Amazon VPC	Isolated network for project resources.

Public subnets (2)	Provide subnet placement for public-facing components such as the ALB and Bastion Host, according to the final design.
Private subnets (2)	Provide private placement for application EC2 instances managed by Auto Scaling.
Internet Gateway	Provides internet routing for public subnet resources when public addressing, routes, and security rules permit it.
Route tables	Control routing for public and private subnets.
Amazon EC2	Compute capacity for Bastion Host and application instances.
Security Groups	Control inbound and outbound traffic to the Bastion, ALB, and application instances.
Application Load Balancer (ALB)	Receives application requests and forwards them to healthy targets.
Target Group	Registers application instances and performs health checks.
Launch Template	Defines the AMI, instance type, security groups, and launch configuration for Auto Scaling.
Auto Scaling Group (ASG)	Maintains the desired number of EC2 instances and can replace unhealthy instances.
Amazon S3 / Route 53	Include only if actually configured in this project; record the bucket or hosted-zone/record details in the appendix.

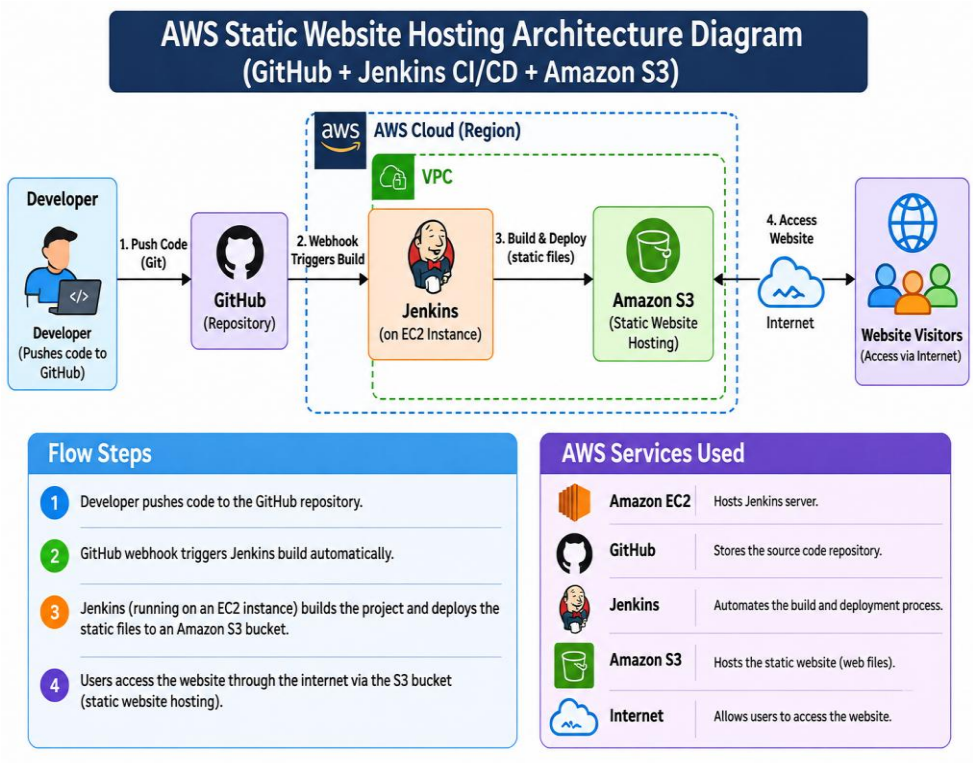
4. Architecture Overview

Logical traffic flow:

```
Users / Browser
    |
    v
Application Load Balancer (public subnets across two AZs)
    |
    v
Target Group + Health Checks
    |
    v
Auto Scaling Group (application EC2 instances in private subnets)

Administrator → Bastion Host (public subnet) → private resources using private IP / approved access
method
```

The ALB should be attached to at least two suitable subnets in different Availability Zones. Application instances should be registered with the target group and placed in private subnets if that matches the actual configuration. The Bastion Host is an administrative access path, not the normal route for customer web traffic. Attach the final architecture diagram to this section.



5. Network Design and Known Resource Details

Resource	Known value / name	Verification notes
VPC	My Project VPC	VPC ID: vpc-01b9fdae82b418aad
VPC IPv4 CIDR	10.0.0.0/16	Confirm in VPC details
Public subnet 1	Public-Subnet-1	Availability Zone: ap-south-1a (known from Bastion placement)
Public subnet 2	Public-Subnet-2	Record actual AZ and CIDR from console
Private subnet 1	Private-Subnet-1	Route table: MyProject-VPC-rtb-private1-ap-south-1a
Private subnet 2	Private-Subnet-2	Route table: MyProject-VPC-rtb-private2-ap-south-1b
Public route table	MyProject-VPC-rtb-public	Default route 0.0.0.0/0 → IGW
Internet Gateway	My-Project-IGW	Known IGW ID: igw-02bd9129be9b634dc
Bastion instance	MyProject-Bastion	Amazon Linux 2023, t3.micro, Public-Subnet-1, public IPv4 enabled
Bastion Security Group	MyProject-Bastion-SG	Verify inbound SSH is restricted to your current public IP
Auto Scaling Group	MyProject-ASG	Activity status was previously shown as Successful; verify current state
ALB / Target Group / Launch Template	[my project-TG /my project-LT]	Copy exact names from EC2 console

6. Step-by-Step Implementation

Step 1 — Sign in and select the Region

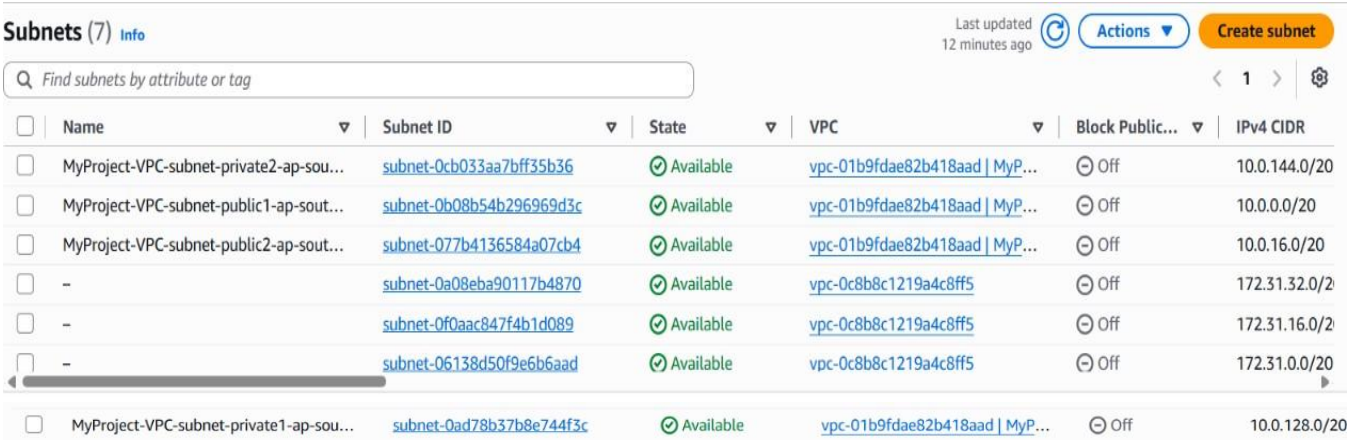
1. Open the AWS Management Console and sign in.
2. Select Asia Pacific (Mumbai), ap-south-1, if this is the Region used for the build.
3. Keep related resources in the same Region unless the architecture requires otherwise.

Step 2 — Create and verify the VPC

4. Open VPC → Your VPCs and locate or create My Project VPC.
5. Set or verify the IPv4 CIDR block 10.0.0.0/16, if that is the actual configuration.
6. Record the VPC ID vpc-01b9fdae82b418aad.
7. Verify the VPC state is Available.

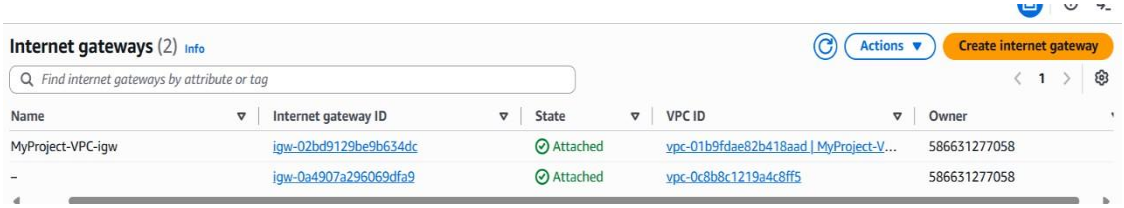
Step 3 — Create four subnets across two Availability Zones

8. Open VPC → Subnets and create or verify Public-Subnet-1 and Public-Subnet-2.
9. Create or verify Private-Subnet-1 and Private-Subnet-2.
10. Place the subnets across two Availability Zones (known private route table suffixes indicate ap-south-1a and ap-south-1b; verify the actual subnet AZs).
11. Ensure subnet CIDR blocks do not overlap and all belong to My Project VPC.
12. Record each subnet's ID, CIDR, and Availability Zone in the appendix.



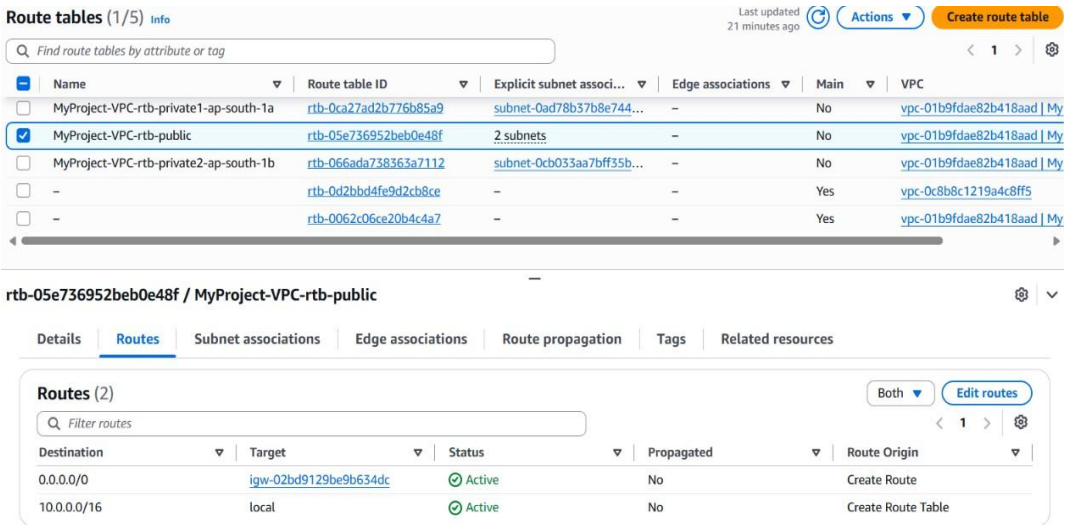
Step 4 — Attach the Internet Gateway

13. Open VPC → Internet gateways and locate My-Project-IGW.
14. Verify it is attached to My Project VPC.
15. Known Internet Gateway ID: igw-02bd9129be9b634dc.
16. If it is not attached, use Actions → Attach to a VPC and select the correct VPC.



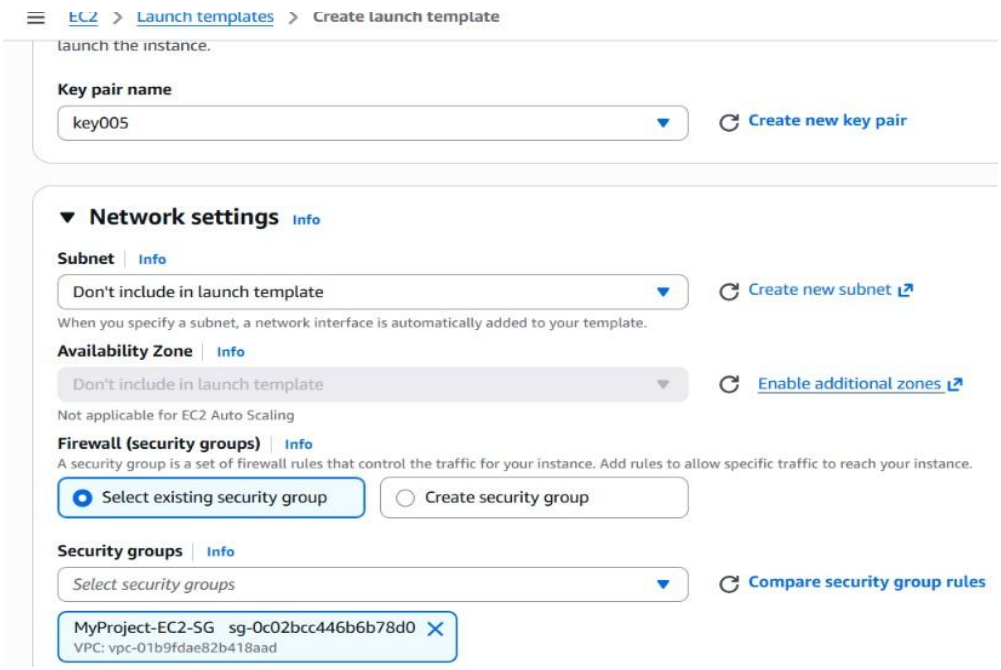
Step 5 — Configure route tables

17. Open Route tables and locate MyProject-VPC-rtb-public.
18. Verify that the table has the local route for 10.0.0.0/16 and a default route 0.0.0.0/0 targeting My-Project-IGW.
19. Verify that Public-Subnet-1 and Public-Subnet-2 are associated with the public route table, if that is the intended architecture.
20. Verify private route tables MyProject-VPC-rtb-private1-ap-south-1a and MyProject-VPC-rtb-private2-ap-south-1b are associated with their respective private subnets.
21. Do not assume private subnets have internet egress: confirm whether NAT Gateway or VPC endpoints were configured. Record the actual route entries.



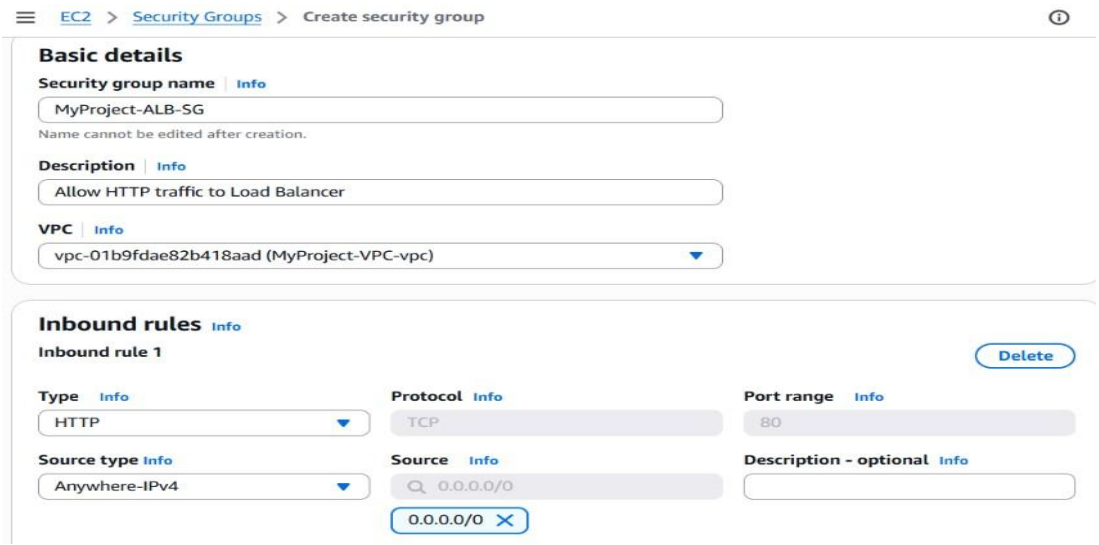
Step 6 — Configure the Bastion Host

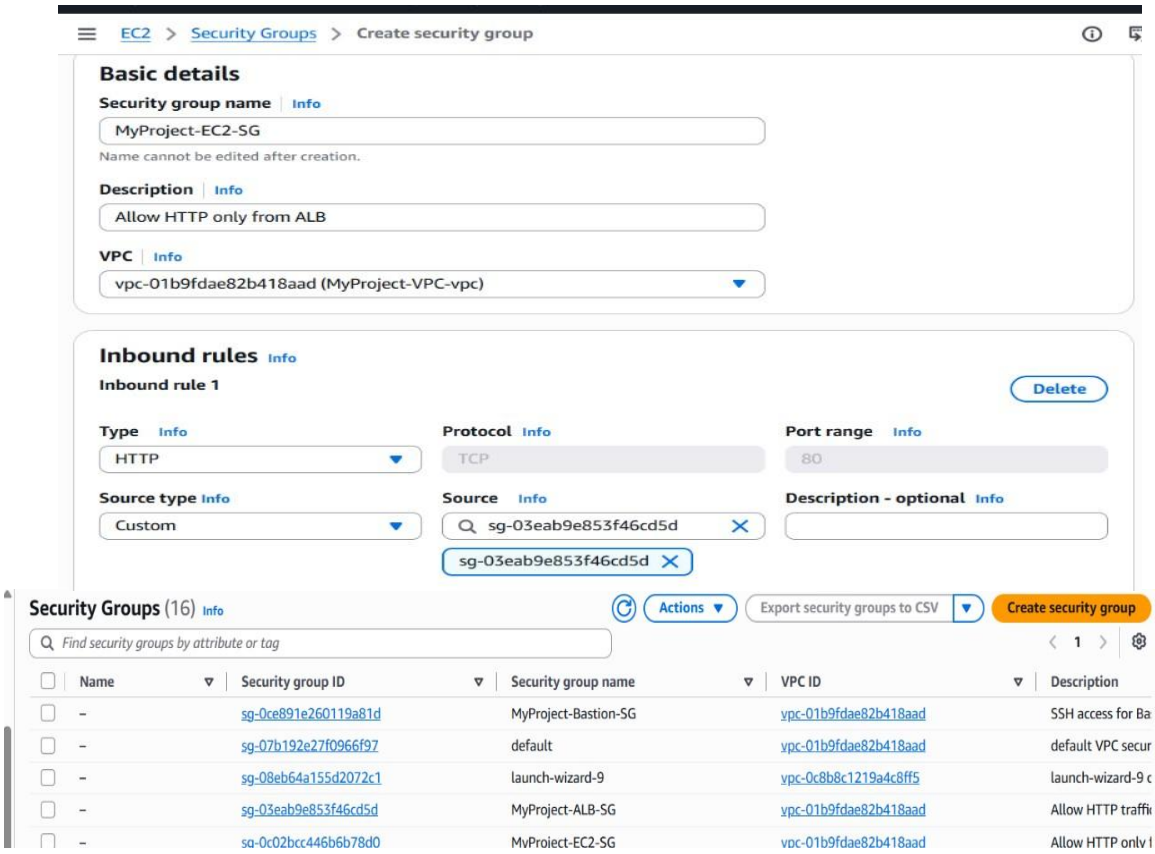
22. Locate or create Security Group MyProject-Bastion-SG in My Project VPC.
23. Restrict inbound SSH (TCP 22) to your current public IP (/32); do not expose SSH to all addresses.
24. Launch or verify EC2 instance MyProject-Bastion.
25. Known settings: Amazon Linux 2023, t3.micro, Public-Subnet-1 in ap-south-1a, public IPv4 enabled.
26. Verify the instance is Running and status checks pass.
27. Record the public and private IP addresses in your private working notes; redact them in public screenshots if appropriate.



Step 7 — Create application Security Groups

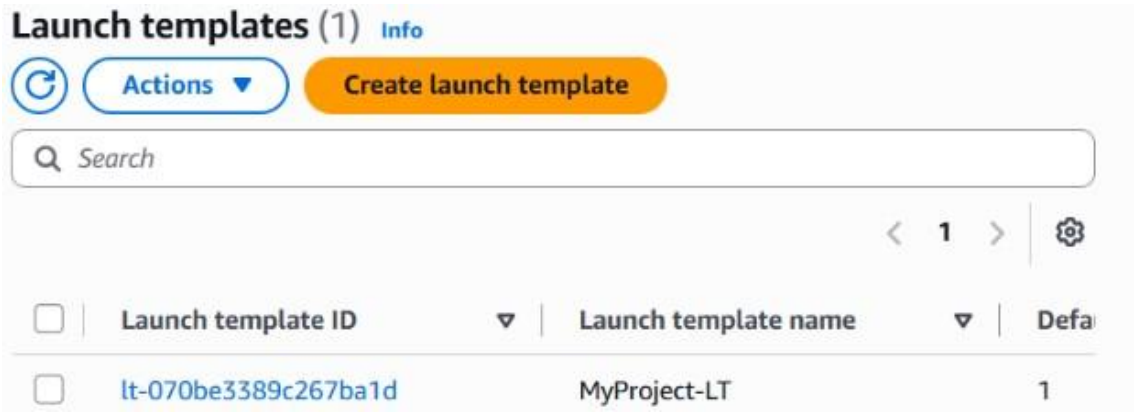
28. Verify or create the ALB Security Group. Allow inbound HTTP (TCP 80) and/or HTTPS (TCP 443) only as required by the application exposure design.
29. Verify or create the application-instance Security Group.
30. Allow the application port from the ALB Security Group, rather than from all internet addresses.
31. If SSH to private instances is required, allow TCP 22 from the Bastion Security Group or use the approved AWS management access method.
32. Remove unnecessary inbound rules and record the final rules in the report.





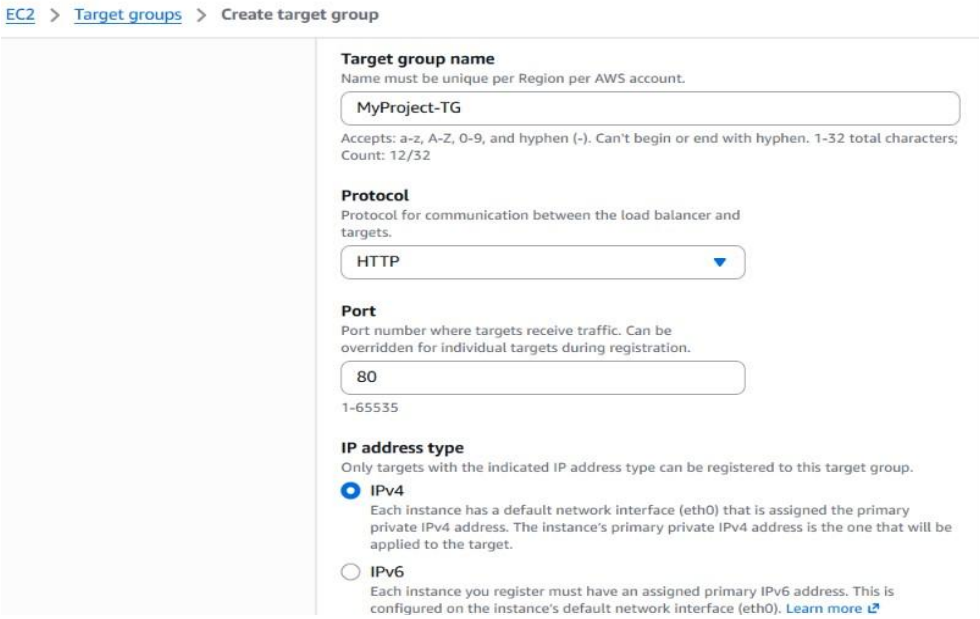
Step 8 — Create or verify the Launch Template

33. Open EC2 → Launch Templates and locate the template used by MyProject-ASG.
34. Record the template name and version.
35. Verify the AMI, instance type, key-pair/access method, instance Security Group, storage, and user data.
36. If user data installs or starts a web application, confirm the commands are correct and the application listens on the port expected by the target group.
37. Do not change a working launch template without checking the Auto Scaling Group and deployment impact.



Step 9 — Configure the Target Group

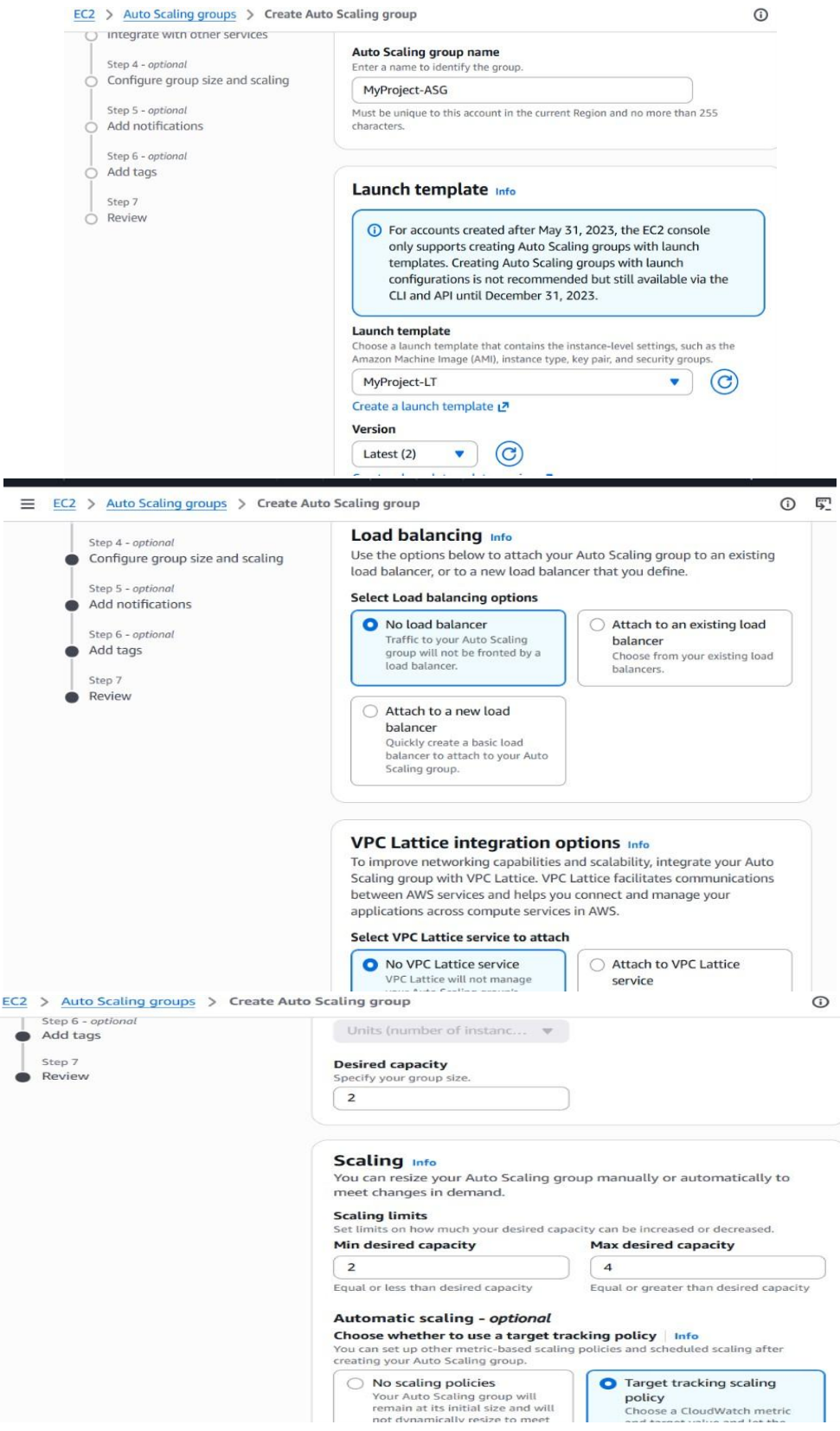
38. Open EC2 → Target Groups and locate the target group associated with the ALB.
39. Record its name, target type, VPC, protocol, and port.
40. Verify the health-check protocol, path, success codes, and interval/threshold values.
41. Verify the Auto Scaling Group uses this target group and instances register as targets.
42. Wait for targets to become Healthy before treating the web tier as ready.

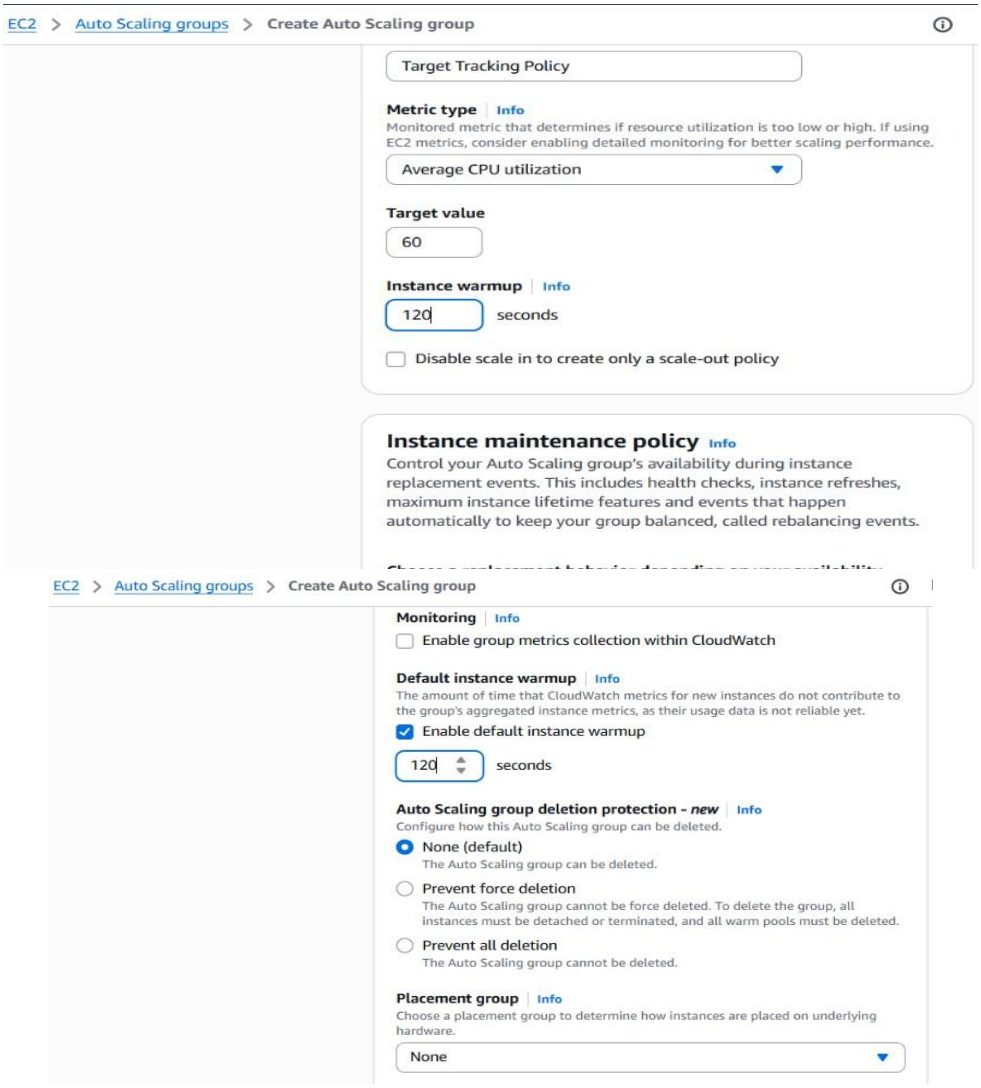




Step 10 — Configure the Application Load Balancer

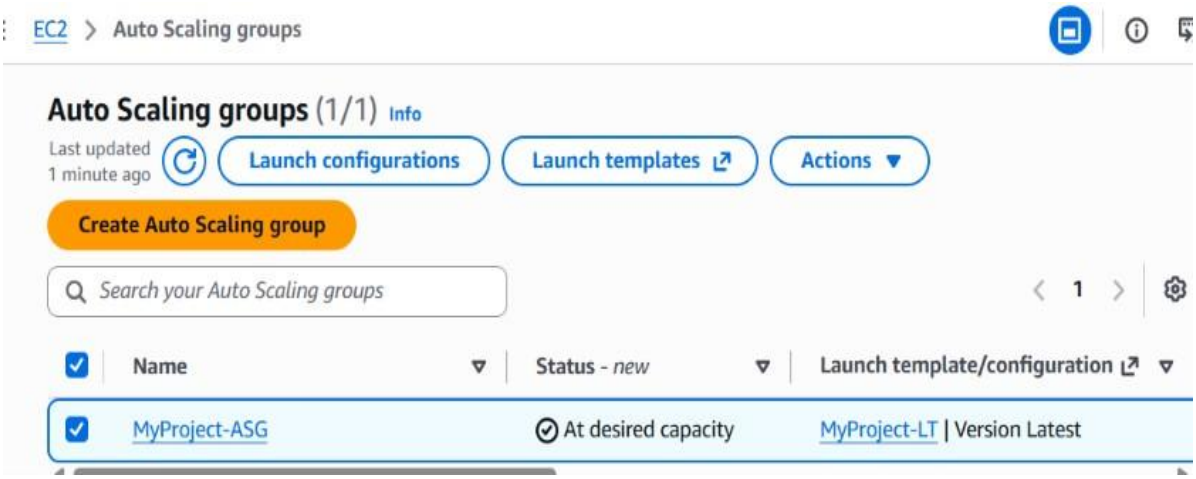
43. Open EC2 → Load Balancers and locate the ALB.
44. Verify it is an Application Load Balancer with the intended internet-facing or internal scheme.
45. Verify it is associated with the intended public subnets in two Availability Zones if internet-facing.
46. Verify the listener (for example HTTP :80 or HTTPS :443) forwards to the correct target group.
47. Verify the ALB Security Group allows the listener traffic and the target-instance Security Group permits traffic from the ALB Security Group.
48. Record the ALB DNS name and test the application through that DNS name.


EC2 > Auto Scaling groups > Create Auto Scaling group
Integrate with other services
Step 4 - optional
Configure group size and scaling
Step 5 - optional
Add notifications
Step 6 - optional
Add tags
Step 7
Review
Auto Scaling group name
Enter a name to identify the group.
MyProject-ASG
Must be unique to this account in the current Region and no more than 255 characters.
Launch template Info
For accounts created after May 31, 2023, the EC2 console only supports creating Auto Scaling groups with launch templates. Creating Auto Scaling groups with launch configurations is not recommended but still available via the CLI and API until December 31, 2023.
Launch template
Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.
MyProject-LT
Create a launch template
Version
Latest (2)
EC2 > Auto Scaling groups > Create Auto Scaling group
Step 4 - optional
Configure group size and scaling
Step 5 - optional
Add notifications
Step 6 - optional
Add tags
Step 7
Review
Load balancing Info
Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.
Select Load balancing options
No load balancer
Traffic to your Auto Scaling group will not be fronted by a load balancer.
Attach to an existing load balancer
Choose from your existing load balancers.
Attach to a new load balancer
Quickly create a basic load balancer to attach to your Auto Scaling group.
VPC Lattice integration options Info
To improve networking capabilities and scalability, integrate your Auto Scaling group with VPC Lattice. VPC Lattice facilitates communications between AWS services and helps you connect and manage your applications across compute services in AWS.
Select VPC Lattice service to attach
No VPC Lattice service
VPC Lattice will not manage
Attach to VPC Lattice service
EC2 > Auto Scaling groups > Create Auto Scaling group
Step 6 - optional
Add tags
Step 7
Review
Units (number of instanc...
Desired capacity
Specify your group size.
2
Scaling Info
You can resize your Auto Scaling group manually or automatically to meet changes in demand.
Scaling limits
Set limits on how much your desired capacity can be increased or decreased.
Min desired capacity
2
Equal or less than desired capacity
Max desired capacity
4
Equal or greater than desired capacity
Automatic scaling - optional
Choose whether to use a target tracking policy Info
You can set up other metric-based scaling policies and scheduled scaling after creating your Auto Scaling group.
No scaling policies
Your Auto Scaling group will remain at its initial size and will
Target tracking scaling policy
Choose a CloudWatch metric



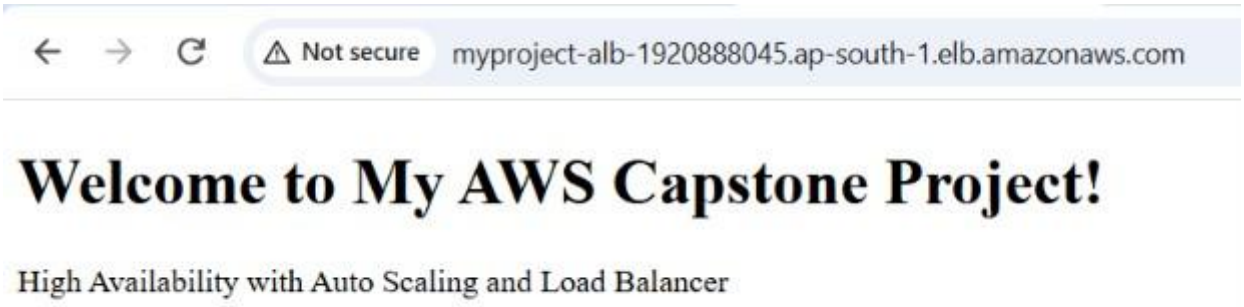
Step 11 — Configure the Auto Scaling Group

49. Open EC2 → Auto Scaling Groups and locate MyProject-ASG.
50. Verify the associated Launch Template and target group.
51. Verify the VPC and intended private subnets across the two Availability Zones.
52. Record the minimum, desired, and maximum capacity values actually configured.
53. Check the Activity tab. The project previously showed a Successful activity result; capture the current activity status and instance health.
54. Confirm instances launch and register in the target group as expected.



Step 12 — Deploy and verify the web application

55. Use the application setup that was actually configured in the Launch Template, user data, or instance deployment steps.
56. Confirm the web server or application process is running on the expected port.
57. Open the ALB DNS name in a browser.
58. Verify the expected web page or application response.
59. Capture a screenshot of the working page and note the date/time and test result.



Step 13 — Optional services: S3 and Route 53

Include this section only for services actually configured in the project. If S3 was used to store website assets or documentation, record the bucket purpose and access policy without publishing secrets. If Route 53 was used, record the hosted zone, DNS record type/name, and its routing target. Do not add these services to the final architecture merely because they are AWS services.

8. Troubleshooting Guide

Problem	Checks and corrective actions
504 Gateway Time-out	Check target health; app process and listening port; health-check path; ALB listener/target-group port; target Security Group source; application logs and response time.
Targets unhealthy	Confirm application responds on the health-check path and returns an accepted status; verify port, protocol,

	Security Groups, and instance startup/user-data logs.
ASG activity failed	Open ASG Activity history; inspect launch-template version, subnet IP availability, instance type availability, AMI, IAM permissions, and Security Group/VPC configuration.
Cannot SSH to Bastion	Check public IPv4, instance status checks, public route to IGW, current client public IP in inbound rule, key file and username.
Cannot reach private instance	Check private instance route/reachability, Security Group reference to Bastion, private IP, SSH authentication, and approved key-forwarding or AWS access method.
Private instance cannot download packages	Check whether a NAT Gateway or required VPC endpoints exist; private subnets do not automatically receive internet access.
Subnet/VPC resource cannot be deleted	Identify and remove dependent resources (instances, ENIs, load balancers, endpoints, NAT Gateway, etc.) in a safe order before retrying.

10. GitHub Repository Structure

```
aws-high-availability-project/
├── README.md
├── documentation/
│   ├── AWS-High-Availability-Project-Report.docx
│   └── architecture.png
└── screenshots/
    ├── vpc.png
    ├── subnets.png
    ├── route-tables.png
    ├── bastion.png
    ├── launch-template.png
    ├── target-group-health.png
    ├── load-balancer.png
    ├── autoscaling-group.png
    └── website-test.png
```

60. Create or open the GitHub repository.
61. Add a README.md containing the overview, architecture, AWS services, implementation summary, tests, and screenshots.
62. Upload this report and the architecture diagram.
63. Upload only sanitized screenshots. Never upload private keys (.pem/.ppk), passwords, access keys, tokens, or secrets.
64. Open the repository after upload and verify all files and image links work.

11. Cost and Cleanup

- Review AWS Billing and Cost Explorer for EC2, ALB, NAT Gateway, EBS volumes, Elastic IPs, S3, Route 53, and data transfer charges as applicable.
- A running ALB and NAT Gateway can incur charges even when the website receives little traffic.
- After capturing evidence, terminate or delete resources that are no longer required, in dependency-aware order.
- Before cleanup, confirm whether the instructor needs the live environment for review.
- Verify that unused billable resources are removed; stopping EC2 alone does not stop all AWS charges.

12. Conclusion

The AWS High Availability Web Application project demonstrates VPC networking, public/private subnet design across Availability Zones, controlled administrative access through a Bastion Host, load balancing, target health checks, and EC2 capacity management using a Launch Template and Auto Scaling Group. The project outcome should be supported by actual AWS console screenshots and a successful test through the ALB DNS name. Only the services and test results verified in the AWS account should be included in the final submission.